#5/#6 If that distance is less than current NN's distance, we have to update the values stored for the NN and its distance.

#7 If, instead, `node` is an internal node, we need to cycle through all its children and possibly traverse their subtrees. We start by sorting `node`'s children from the closest to the furthest WRT target. As mentioned, a different heuristic than the distance to the bounding envelope can be used here.

#8 Cycle through all children, in the order we sorted them.

#9 Check if their bounding envelope intersects with the NN bounding sphere: in other words, if the distance from `target` to the closest point within `child`'s bounding envelope is smaller than the pruning distance (`nnDist`).

#10 If the distance between `target` and `child` is smaller than pruning distance, then traverse the subtree rooted at `child`, updating the result.

#11 We only have left to return the updated values for the best result found so far.

Of the helper methods in listing 10.19, it's important to spend some time explaining function `nodeDistance`, described in listing 10.20. If we refer to figure 10.22, you can see why the minimum distance between a node and a bounding envelope is equal to the distance between the envelope's centroid minus its radius: we just used the formula for the distance between a point and a sphere, taken from geometry.